EXPERIMENTAL DATA MANAGEMENT SYSTEM

MILESTONE 3 AND 4 DOCUMENTATIONS

1. INTRODUCTION

This document outlines the development, improvements, and challenges encountered in Milestones 3 and 4 of the Experimental Data Management System.

The goal of these milestones was to transform the system from a basic object-oriented structure into a data-driven, robust and maintainable system capable of handling real world experimental data.

2. MILESTONE 3: DATA STRUCTURES AND FUNCTIONAL ABSTRACTION

2.1. Objectives

- Integrate a real-world dataset
- Use advanced data structures
- Apply functional programming concepts
- Implement efficient data processing workflows
- Enable file handling and persistence

2.2. What changed

1. Integration of Real Dataset

We introduced dataset loading using pandas.

Data is read from a CSV file using:

pd.read_csv(file_path, sep=';')

Impact: The system became dynamic and capable of handling real-world structured data instead of hardcoded values.

2. Dynamic Measurement Representation

Replaced fixed variables (e.g., temperature) with: variable, value, unit

Impact: The system can now process any dataset structure, making it flexible and scalable.

3. Functional programming

Introduced lambda and filter() for validation

Impact: Improved code conciseness and readability.

4. Generators

Implemented measurement_generator() using yield

Impact: Efficient memory usage when handling large datasets.

5. Advanced Data Structures

Set – for storing unique variables

Queue(deque) – for managing tasks in FIFO order

Impact: Demonstrates structured and efficient data handling techniques.

6. Statistical Analysis

Computed mean, variance, using pandas grouping functions.

Impact: Enabled meaningful insights from experimental data.

7. File Handling

Implemented CSV export functionality

Impact: Allows processed data to be stored and reused.

2.3. What Failed

Displaying Full Dataset: the system attempted to print all trials. This caused the output to start at a higher number (i.e., Trial 1536). Earlier trials were not visible

2.4. Why It Failed

Terminal output has limited display capacity. Large dataset caused earlier outputs to scroll out of view

2.5. How it was Fixed

Modified the display method to limit output by putting a limit of 10 Trials. As a result, the output became readable and presentable.

3. MILESTONE 4: ROBUSTNESS, EXCEPTIONS & SYSTEM DESIGN PATTERNS

3.1. Objectives

- Improve system reliability
- Add user interaction
- Implement design patterns
- Enhance error handling
- Support structured data storage

3.2. What changed

1. Exception Handling – introduced try/except blocks.

Impact: prevents system crashes and ensures graceful error handling

2. Custom Exceptions – introduced exception classes; DataLoadError and ValidationError.

Impact: provides clearer and more meaningful error messages.

3. Logging System – logs are written to log.txt with timestamps

Impact: Tracks system activity for debugging and auditing.

4. Command-Line Interface (CLI) – added menu-based interaction using input ()

Impact: Improved usability and made the system interactive.

5. Strategy Pattern (Validation) – separated validation logic into independent classes.

Impact: Allows flexible validation rules without modifying core classes.

6. Observer Pattern (Logging) – introduced automatic notification system which notifies the entire system when changes occur.

Impact: decouples logging from system logic and improves modularity.

7. JSON Serialization; json.dump(data, file, indent=4)

Impact: enables structured and portable data storage.

8. Data Validation – when the dataset contained missing values a validation error was raised.

Impact: Improves data integrity and reliability.

3.3. What Failed

- File Not Found – system crashed when invalid file path was entered
- Missing or Invalid Data – some dataset values were empty or undefined

3.4. Why It Failed

- No validation for file existence
- No checks for missing or invalid values

3.5. How It Was Fixed

Introduced Exception Handling for File Loading and Data Validation.

4. CONCLUSION

Milestones 3 and 4 significantly enhanced the experimental Data Management System by transforming it into a:

- Data-driven system
- Robust and fault-tolerant application
- Interactive and user-friendly platform
- Scalable and maintainable architecture

The system is now capable of handling real experimental data efficiently while maintaining flexibility and reliability.